

SCSE 

Stablecoin Clearing & Settlement Engine

Sivasubramanian Ramanathan

*Product Owner | Fintech & Innovation
Ex-BIS Innovation Hub Singapore*

 **Seeking Opportunities in Singapore**

I am looking for roles in **Product Management, Fintech, Payments, RegTech,** and **Digital Assets.**

"I am not just a Product person. **I build.**"

I have worked across product delivery, user research, and cross-agency collaboration. I enjoy solving complex problems and bringing structure to early ideas.

I care deeply about building products that create real impact.

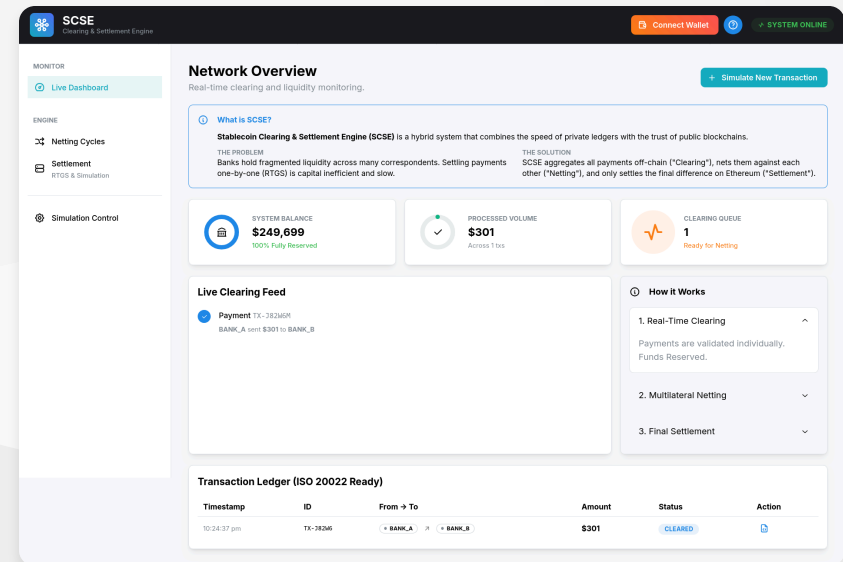
The Learning Journey: SCSE

Goal: To build a real-world Defi application from scratch to learn full-stack Web3 development.

Live Dashboard

I built this React workstation to visualize the invisible logic of clearing engines.

- **Concepts Learned:** State Management (Zustand), React Hooks, Wagmi integration.
- **Outcome:** A working simulation of "Real-Time Gross Settlement" vs "Netting".



My First Smart Contracts

I moved beyond theory and deployed my first Solidity contracts to **Sepolia Testnet**.

1. Settlement.sol

- **Function:** `settleBatch(debtors, amounts, creditors, amounts)`
- **Logic:** Atomically collects from debtors and pays creditors. Reverts if *any* transfer fails (All-or-Nothing).
- **Security:** `onlyOwner` modifier ensures only the Clearing Engine can trigger movement.

2. MockUSDC.sol

- **Standard:** ERC-20 implementation using **OpenZeppelin**.
- **Feature:** Added a public `mint()` function to simulate a faucet for testing.

“**Tools Used:** Hardhat (Compiling/Deploying), Alchemy (RPC), Etherscan (Verification).”

The Problem: Digital Money is Siloed

In my work analyzing **Cross-Border Payments & CBDCs**, I've seen how liquidity fragmentation kills efficiency.

1. **● Trapped Liquidity:**

Stablecoins sit idle in different chains and wallets, requiring massive pre-funding.

2. **● Slow Finality:**

Traditional blockchains (15s - 10 min) are too slow for high-frequency point-of-sale payments.

3. **● High Gas Costs:**

Settling *every* coffee purchase on-chain is inefficient.

“**Goal:** Build a **Hybrid Infrastructure** that combines database speed with blockchain finality.

”

Definitions: Clearing vs Settlement

1. Clearing (Off-Chain)

- **What:** Calculating obligations.
- **Action:** "I owe you \$50".
- **Process:**
 - Validation & Compliance (Travel Rule).
 - Internal Ledger Updates.

2. Settlement (On-Chain)

- **What:** Extinguishing obligations.
- **Action:** "Here is the \$50 token".
- **Mechanism: ERC-20 Transfer.**
 - Blockchain finality = Settlement.

“**Why separate them?:** Netting off-chain turns 1,000 payments into 1 Settlement Transfer.

”

The Problem: The Liquidity Trap

Question: "Stablecoin settlement is just a transfer, why build a Clearing Engine?"

Answer: If you settle every payment individually (Gross Settlement), you need massive pre-funded capital.

Scenario: Gross Settlement (No Engine)

- Alice sends **\$1M** to Bob (Needs \$1M pre-funded).
- Bob sends **\$1M** to Alice (Needs \$1M pre-funded).
- **Total Liquidity Locked: \$2M.**

“Inefficiencies like this are why traditional finance uses Clearing Houses.

”

The Solution: Netting & Stablecoins

The Engine calculates the net obligations *before* moving any tokens.

Scenario: With Clearing Engine

- Engine calculates: Alice +\$1M, Bob -\$1M. THEN Bob +\$1M, Alice -\$1M.
- Net Obligation: **\$0**.
- **Total Liquidity Locked: \$0.**

“***"Where is the Stablecoin part?"***

*In a perfect circle (\$0 net), the Stablecoin is the **Unit of Account** (Measuring debt).*

*In a partial circle (e.g. \$1M vs \$900k), the **Residual Difference (\$100k)** is settled on-chain.*

”

The Deep Dive: Identity vs Value

The core architecture challenge in compliant crypto-finance is separating **Who** from **What**.

Identity (Off-Chain)

- **Standards:** IVMS101 (Travel Rule).
- **Carrier:** Encrypted P2P Messaging (VASP-to-VASP).
- **Content:** PII, Sanctions Data, Source of Funds.
- **Why?:** Privacy (GDPR) & Scalability.

Value (On-Chain)

- **Standards:** ERC-20 / Native Token.
- **Carrier:** Public/Private Blockchain.
- **Content:** Amount, Addresses, Proof of Ownership.
- **Why?:** Finality, Trustlessness, Immutability.

“**Concept:** The Blockchain settles value; VASPs exchange identity off-chain.

”

Why not just ISO 20022?

ISO 20022 is the gold standard for banking (SWIFT MX), but it has limitations in the crypto ecosystem:

1. **Bank-Centric Model:** Assumes counterparties are regulated FIs (BICs) with accounts. In crypto, "Self-Custody" (Unhosted Wallets) exists.
2. **Privacy Leaks:** Rich data payloads (Remittance Info) on a transparent blockchain = Doxing users.
3. **Complexity:** ISO messages are heavy. Crypto needs lightweight, purpose-built identity packets.

The Hybrid Compromise

- **Internal Ops / Fiat Legs:** Use **ISO 20022**.
- **VASP-to-VASP Messaging:** Use **IVMS101**.

IVMS 101: The Identity Standard

IVMS 101 (InterVASP Messaging Standard) is the data model for the "Travel Rule".

- It is **NOT** a transport protocol (it's the payload, not the pipe).
- It is **NOT** XML/JSON specific (though usually JSON).

What's Inside?

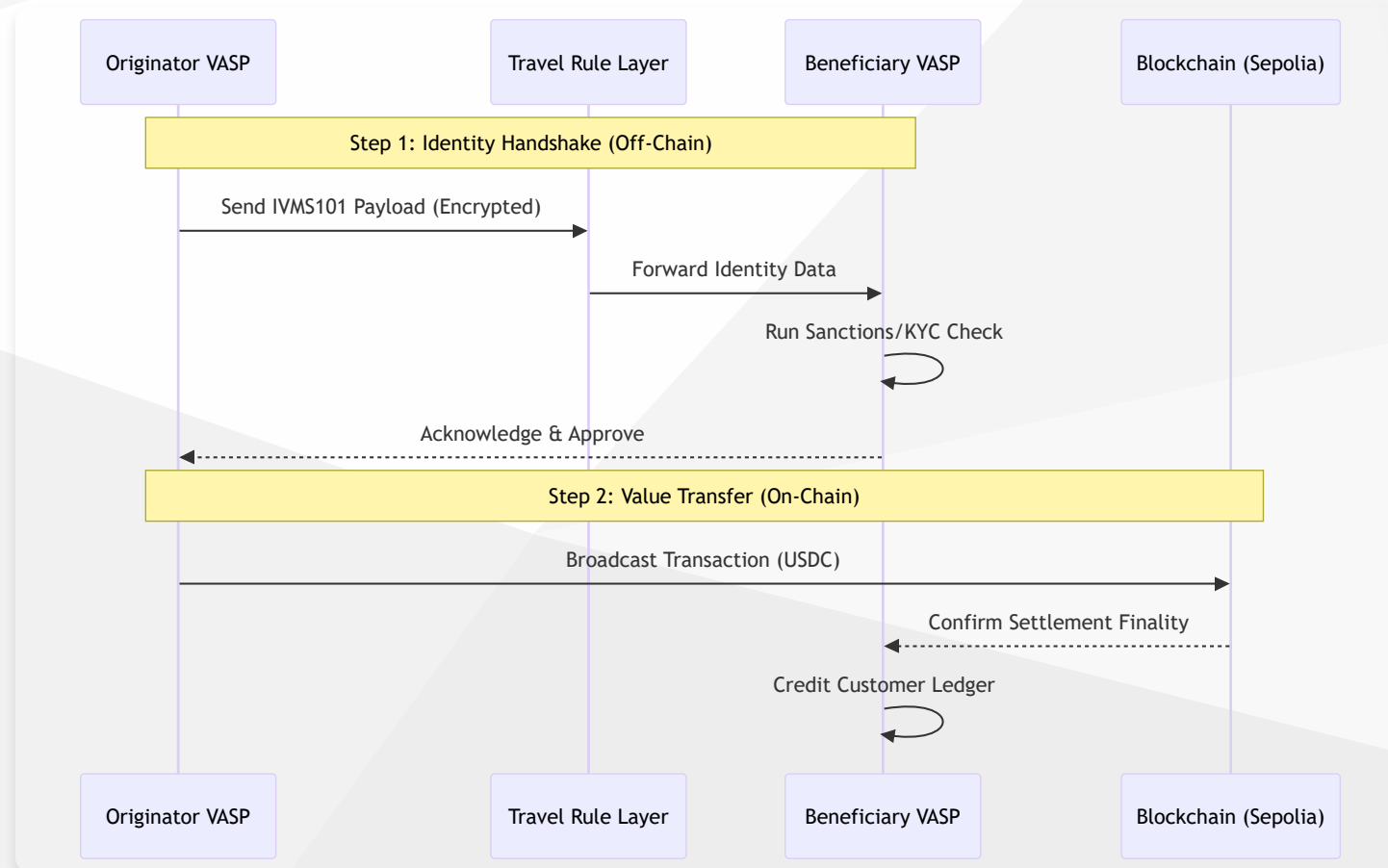
- **Originator**: Name, Account Number (Wallet Address), Physical Address, National ID.
- **Beneficiary**: Name, Account Number.
- **Beneficiary VASP**: Identification code (LEI, BIC).

*“**Analogy**: IVMS101 is the "Envelope" containing the ID cards. The Blockchain is just the "Truck" moving the cash.*

”

The Compliant Flow (Custodial)

How a compliant stablecoin transfer *actually* happens.



The Solution: SCSE

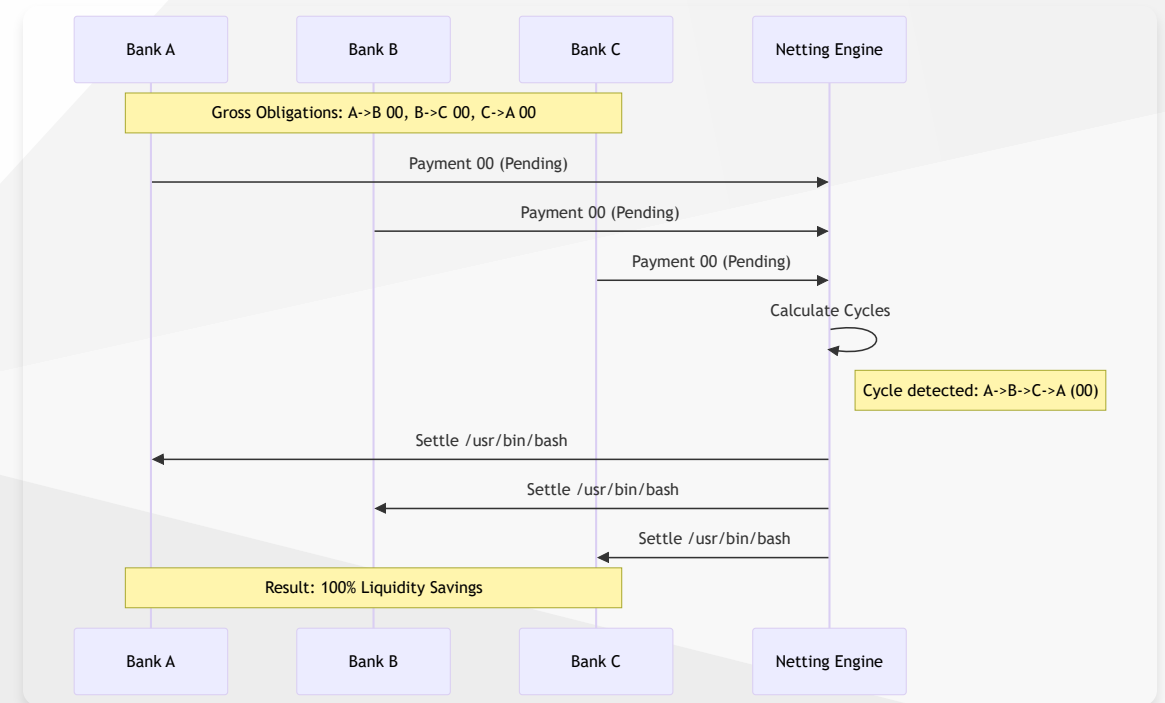
A portfolio-grade financial engine that implements **Real-Time Gross Settlement (RTGS)** and **Multilateral Netting** for Stablecoins.

Hybrid Architecture

- ⚡ **Layer 1 (Off-Chain):**
10k+ TPS. Immediate clearing. Privacy-preserving compliance.
- 💰 **Layer 2 (On-Chain):**
Sepolia Testnet. Cryptographic proof of settlement.

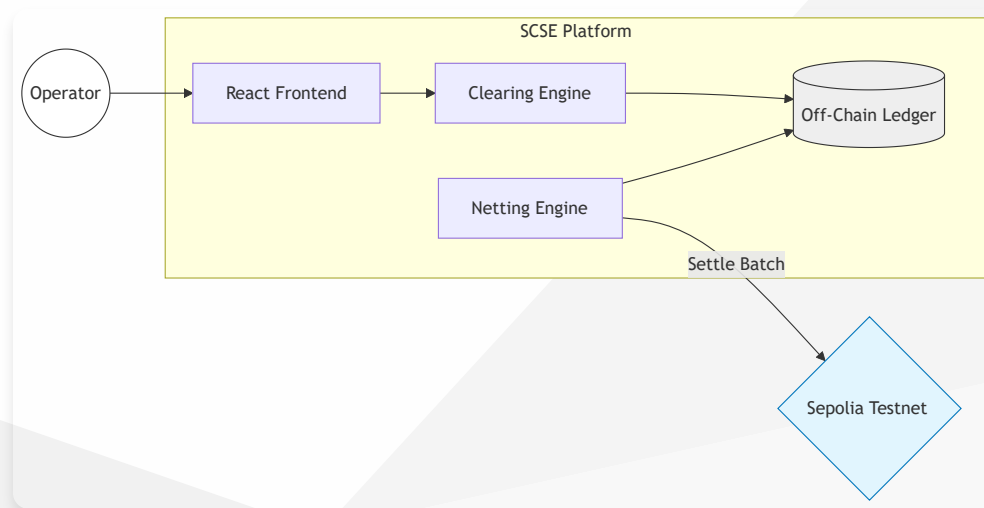
100% Liquidity Savings

Algorithms identify and resolve circular debts (A->B->C->A).



Technical Architecture

I built a full-stack simulation to demonstrate end-to-end payment lifecycles.



- **Frontend:** React, Mantine, Zustand.
- **Web3:** Wagmi, Viem, Solidity (Hardhat).
- **Logic:** Python (FastAPI/SQLAlchemy) adapted to client-side logic for the demo.

Engineering Philosophy & Impact

This project reflects my ability to bridge **Policy** and **Code**:

1. **Compliance-First:**

Built with hooks for **Travel Rule (IVMS101)** compliance, ensuring regulatory readiness.

2. **Standards-Based:**

Uses **ISO 20022** terminology (DebtorAgent, CreditorAgent) for interoperability.

3. **Real-World Reliability:**

Implements proper **Double-Entry Accounting** logic, not just simple token transfers.

About the Builder

Sivasubramanian Ramanathan

Product Owner | Fintech, RegTech & Digital Innovation
PMP | PSM II | PSPO II

I specialize in taking messy, real-world complexity and structuring it into reliable products.

Open for roles that sit between policy, technology, and stakeholder engagement.

Lets Connect

I am ready to bring this level of engineering rigor and product thinking to your team.

-  **Portfolio:** sivasub.com
-  **LinkedIn:** linkedin.com/in/sivasub987
-  **Code:** github.com/siva-sub/Stablecoin-Clearing-and-Settlement-Engine
-  **Docs:** [Full Documentation in /docs](#)

Live Demo:

siva-sub.github.io/Stablecoin-Clearing-and-Settlement-Engine